Here is a simplified look at the steps to making a CORS request (there are a few more nuances to some CORS requests, which we'll cover in later chapters):

1. The CORS request is initiated by the JavaScript client code.
2. The browser includes additional HTTP headers on the request before sending the request to the server.
3. The server includes HTTP headers in the response that indicate whether the request is allowed.
4. If the request is allowed, the browser sends the response to the client code.

If the headers returned by the server don't exist, or aren't what the browser expects, the response is rejected and the client can't view the response. In this way, browsers can still enforce the same-origin policy on servers that don't allow cross-origin requests. Now that you have a sense of what CORS is, let's turn our attention to making a CORS request.

1.2 CORS by example

Let's demonstrate how CORS works by building a Flickr sample app. Figure 1.2 shows the app, which loads photos from the NYPL's Flickr site and displays them on the page.

The following listing shows the code behind this sample.

Listing 1.1 Making a CORS request

```
<!DOCTYPE html>
<html>
<body onload="loadPhotos();">
<div id="photos">Loading photos...</div>
<script>
function loadPhotos() {
  var api_key = '<YOUR API KEY HERE>';
  var method = 'GET';
  var url = 'https://api.flickr.com/services/rest/?' +
      'method=flickr.people.getPublicPhotos&' +
      'user_id=32951986%40N05&' +
      'extras=url_q&format=json&nojsoncallback=1&' +
      'api_key=' + api_key;

  var xhr = new XMLHttpRequest();
  if (!('withCredentials' in xhr)) {
    alert('Browser does not support CORS.');
    return;
  }
  xhr.open(method, url);

  xhr.onerror = function() {
    alert('There was an error.');
  };

  xhr.onload = function() {
    var data = JSON.parse(xhr.responseText);
    if (data.stat == 'ok') {
      var photosDiv = document.getElementById('photos');
      photosDiv.innerHTML = '';
```

Request to Flickr API

Makes sure browser supports CORS